

Project Overview (Plain-English)

What this is

A small web app for learning Japanese. You open an HTML page in a browser and it shows lesson steps. You type romaji (like 'kon'nichiwa') and the app turns it into kana (■■■■■). It can also speak lines, and in some browsers listen to your voice.

What lives where

HTML pages (the 'screens' of the app):

- Kana-reader2/index.html — title: 'KanaReader', scripts: 0
- Kana-reader2/lesson.html — title: 'KanaReader — Lesson', scripts: 4
- Kana-reader2/old lesson attempts/back-up-index.html — title: 'KanaReader', scripts: 8
- Kana-reader2/old lesson attempts/reference.html — title: 'Japanese Kana Reference', scripts: 0

JavaScript (the brains):

- Kana-reader2/lesson-loader.js — about 2879 bytes; notable functions: loadLessons, pickLesson, show, updateTitle, waitForShim
- Kana-reader2/lesson-shim backup copy.js — about 54079 bytes; notable functions: Speech, attachGreetingNormalizer, attachKanaGuide, attachSmartNormalizer, bindNDisambiguator, bindOnce
- Kana-reader2/lesson-shim.js — about 54802 bytes; notable functions: Speech, attachGreetingNormalizer, attachKanaGuide, attachSmartNormalizer, bindNDisambiguator, bindOnce
- Kana-reader2/memorymatch.js — about 4243 bytes; notable functions: checkForMatch, onCardClick, resetTurn, shuffle, speakKana, startMemoryMatch
- Kana-reader2/sw.js — about 2510 bytes; notable functions: functions defined inside this file
- Kana-reader2/modules/ninjaSlice.js — about 7584 bytes; notable functions: draw, drawTrails, endGame, initNinjaSlice, resize, sliceAt
- Kana-reader2/old lesson attempts/backup copy of js script.js — about 41279 bytes; notable functions: addLine, appendChatMessage, bindLineEvents, checkForMatch, converterSpeak, drawFrame
- Kana-reader2/old lesson attempts/backup copy of ninjaslice.js — about 12008 bytes; notable functions: drawFrame, drawTrails, endGame, getPointerPosition, handleSlice, initNinjaSlice

CSS (the look):

- Kana-reader2/style.css — selectors: #lesson-main, #lesson-row, #mascot

Data files / manifest:

- Kana-reader2/manifest.webmanifest — top-level keys: name, short_name, lang, start_url, scope, display, orientation, background_color, theme_color, icons
- Kana-reader2/stories.json — top-level keys: lessons
- Kana-reader2/images/site.webmanifest — top-level keys: name, short_name, icons, theme_color, background_color, display
- Kana-reader2/old lesson attempts/stories-backup.json — top-level keys: (could not parse)

How typing works (simple)

When you type romaji, the app uses a library called WanaKana to convert it into kana. Example: typing 'kon\nichiwa' becomes こんにちは. The code also adds some small helpers: it corrects common mistakes (like ん vs の), handles small の, and compares what you typed to the answer using kana (so punctuation like ! won't block you).

Audio and voice

Text-to-speech (TTS) uses the browser's built-in voices. It works on most devices, but the first sound usually requires a tap. Speech recognition (listening) only works on some browsers like Android Chrome. iOS Safari does not support it; the app shows a friendly fallback.

Deploying (very short)

1) Put your files in a GitHub repo. 2) Enable GitHub Pages to host them. 3) On your phone, open the Pages link and test typing + sound. 4) For an Android app, turn the site into a PWA and use Bubblewrap to package it.

Glossary (super short)

- Relative path: 'stories.json' (works from any folder), not '/stories.json' (root-only).
- PWA: website that can be installed like an app.
- TTS: computer voice that reads text.